

HAIRSPRING: Models propose; software verifies

An agent harness that checks work, records evidence, recovers after crashes, and tests its own improvements

Tensorbend Instinct

September 2026

Abstract

A language model can write code, use tools, and say that a task is finished. Its confidence is not proof. Hairspring separates the worker's proposals from executable checks, a fresh-context review, a durable event record, and separately validated shared state and policy changes. This paper describes those mechanisms and their failure boundaries. A new paired comparison with OpenHands on a locked 15-instance SWE-bench Verified sample, using DeepSeek V4.1 Flash on both sides, is in preparation. Offline harness and scorer validation is complete, but no pair has a completed paid result; this revision reports the method without claiming a winner. The former ten-task comparison used different conditions and cannot be used as evidence for the new comparison.

1 The problem in plain English

Most software agents repeat the same cycle: give a model a goal, let it use tools, show it the results, and stop when it submits an answer [1, 3]. This works until the model is wrong about being done. It may run the wrong test, repair only the visible symptom, forget an earlier failure, or report success after a tool broke.

Hairspring separates doing the work from deciding what becomes official. The model may search, edit, run tools, start limited child agents, and revise its answer. It may not write the final pass result. Software checks run against the submitted files. A second model, started with fresh context, may look for a specific blocking defect, but it cannot override a failed test. One durable event record stores the evidence needed to inspect or resume the run.

The same rule applies beyond the final answer. An agent may propose a shared fact, a memory, or a change to its own instructions. Separate software decides whether to accept it. Software makes the final decision on whether each proposal becomes official. Figure 3 pairs each proposal with the software that makes that decision.

What this paper contributes.

1. A plain comparison of five common agent designs and the unchecked failure each leaves open.
2. A complete mission path in which tests and a fresh review decide whether work passes, while one event record supports inspection and crash recovery.
3. Separate acceptance paths for shared state, memory, and changes to the harness.
4. A locked, paired 15-instance evaluation protocol with explicit native-grader and unfinished-run boundaries; outcome tables remain pending.

2 Five common agent designs

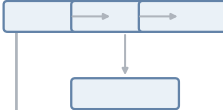
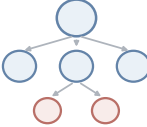
Figure 1 is a map of the field. The categories describe control-flow shapes, not named products. Real systems often combine them.

Direct loop. One model repeats think, act, and observe, then submits when it believes the task is done [1, 3, 4, 2]. This is simple and cheap. It also lets the worker grade itself, so a fluent wrong answer can cross the finish line [5].

Fixed workflow. A person draws a graph of model steps and branches before the run starts [13]. This is predictable for known cases. A new situation may have no valid route, and a model-based checking step is still a model opinion unless an external test owns the result.

Five agent designs and their unchecked authority

Each class gets work done. Each keeps some of five authorities (close, record, share, remember, improve) in the model that does the work.

C1. Direct loop	C2. Static workflow graph	C3. Role ensemble	C4. Search and evolution	C5. Memory-augmented
one model in a think-act-observe loop; it submits when it says it is done	human-authored DAG of LLM steps; edges fixed before the run begins	copies of one model debate, vote, or play roles	tree search or prompt evolution scored by the model itself	retrieval store or summaries re-injected into the prompt
				
kept in-model	kept in-model	kept in-model	kept in-model	kept in-model
close record share remember improve	close record share, remember, improve: no mechanism at all	close share record remember, improve: no mechanism at all	close improve record, share, remember: no mechanism at all	remember close record, share, improve: no mechanism at all
where it breaks The model grades its own work. Fluent failures cross as passes.	where it breaks Off-script states have no route. The judging node is the same model kind.	where it breaks Correlated errors vote together. Consensus is not correctness.	where it breaks Search optimizes the model's own confidence. Cost grows per branch.	where it breaks Unvalidated writes persist. Retrieval relevance is not correctness.

Shared root cause

In every class the component that produces the work also holds authority over it: it declares the result, owns the history, writes shared state, decides what persists, or promotes its own changes. Model confidence substitutes for execution truth, and every added safeguard is one more opinion from the same source.

Five authorities move out of the model and into a replayable substrate.

Class evidence is cited in the bibliography; the argument here is about shapes, not products.

Figure 1: Five common agent designs. The middle column says what each design does. The right column states the unchecked failure that remains. All five can do useful work; none, by shape alone, prevents a model's own mistake from becoming the accepted result or lasting state.

Model team. Several copies of a model plan, debate, vote, or take roles [6]. Different wording is not independent evidence. Shared blind spots can win a vote, producing agreement without correctness [7, 8].

Search or evolution. A controller generates many reasoning paths, prompts, or policies and selects the highest-scoring candidate [9, 10, 11]. This can be strong, but it fails when a candidate influences the judge or the model's confidence becomes the score. Cost also grows with the number of branches and trials.

Memory add-on. A store retrieves notes from earlier runs and places them in the next prompt. This extends a model across sessions. Without validation and source tracking, a wrong note can become a lasting fact; relevance does not mean truth [12].

Shared problem. In each shape, some model output can become accepted without a different mechanism proving it. Hairspring addresses that problem by giving each kind of proposal a separate acceptance path.

3 What must be decided outside the worker

Five questions appear in a long-running agent system. Each has a plain software owner.

Did the work pass? Executable checks run on the submitted workspace. A fresh reviewer may block a result by naming a grounded defect.

The Hairspring pipeline: the model proposes, the substrate decides

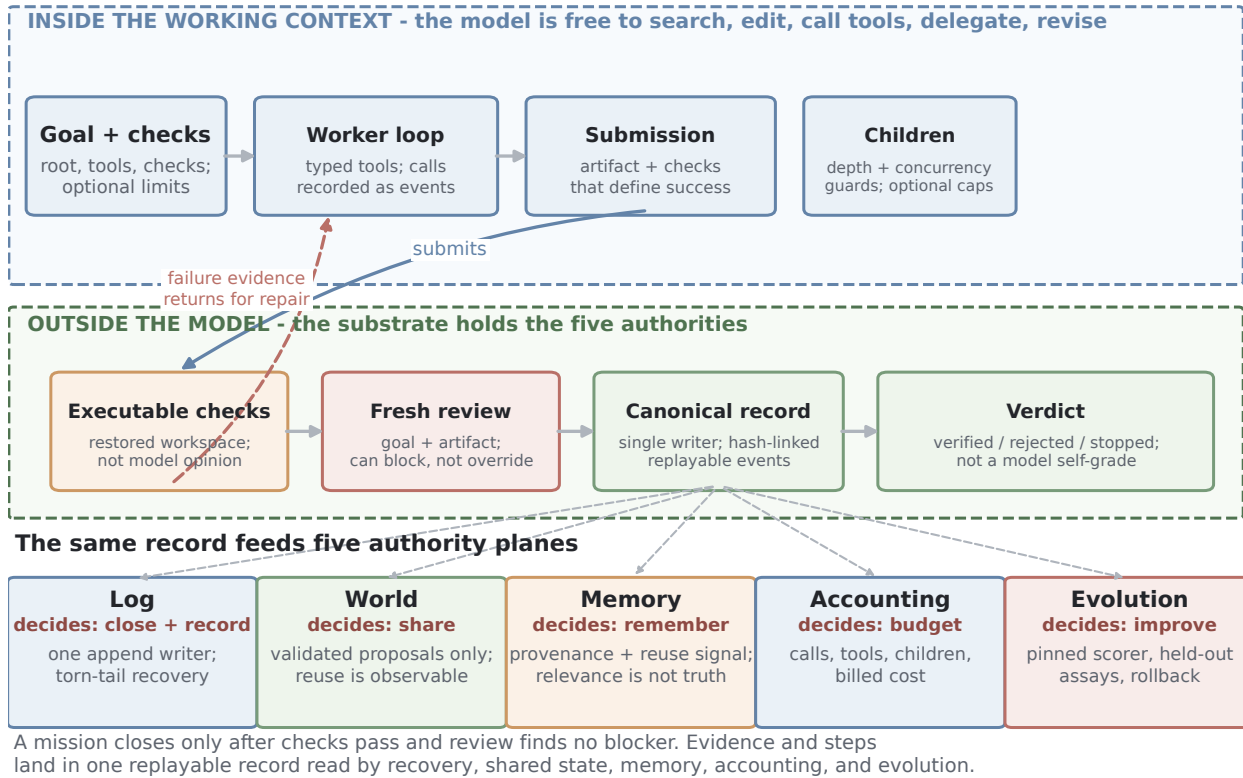


Figure 2: One Hairspring mission. Read left to right across the top. Failures loop back to the model as repair evidence. Every downward arrow writes to the same run record, which stores the work, checks, review, cost, and final outcome.

What happened? A single event writer stores ordered model calls, tool results, file identities, costs, and verdicts.

What may agents share? A world service validates proposed shared state before other agents can treat it as accepted.

What should be remembered? A memory record keeps its source and later-use signal. A retrieved note is never relabeled as proven truth.

Did a harness change help? A scorer fixed before the trial compares old and new policies on held-out tasks and records why a change was kept or undone.

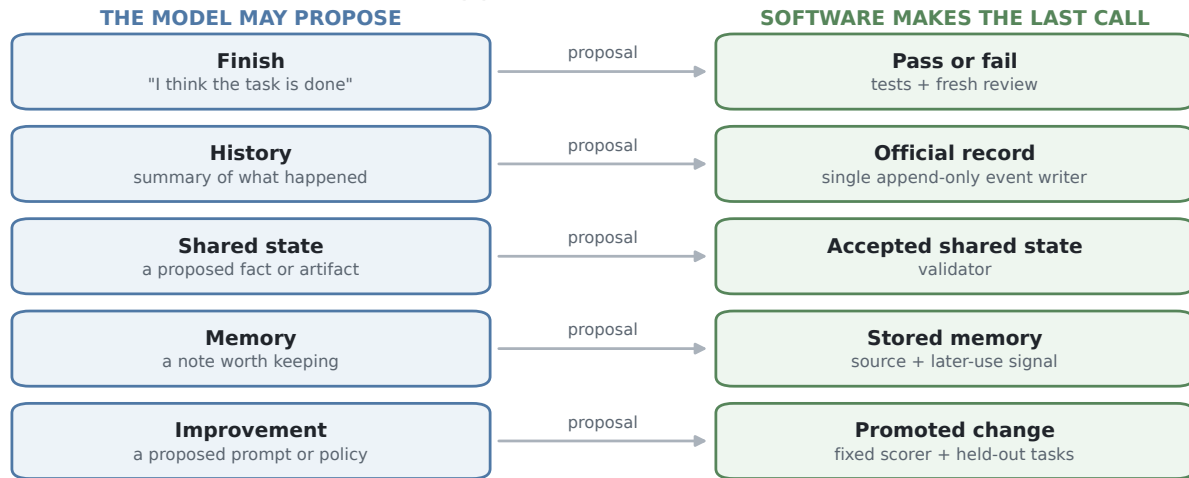
The worker can still be creative. These boundaries govern only what becomes a pass, official history, shared state, stored memory, or an installed policy.

4 A mission from start to finish

A mission begins with a goal, a project directory, available tools, and success checks. Step, wall-time, and mission spending limits are optional and are not set for the paired evaluation. The model works through typed tools. Each model call and tool result is recorded. When ready, the model submits both an artifact and the checks that should prove it.

The checker restores the workspace to the submitted state and runs those checks there. If a check fails, the failure evidence goes back to the worker for repair. If the checks pass, a fresh-context reviewer sees the goal and artifact, without the worker's explanation, and looks for a blocking defect. That reviewer can return a defect; it cannot turn a failed check into a pass. Only a clean check result plus no blocking finding creates a verified outcome.

Who decides what: model suggestions versus software decisions



The model can suggest each outcome. A separate mechanism decides what becomes official.

Figure 3: Who makes the last call. Each row pairs a model suggestion with the software decision required before it becomes official. Software makes the final decision on whether each proposal becomes official.

5 The boundary between suggestions and decisions

Figure 3 shows the boundary directly. The left side lists what a model may suggest. The right side names the mechanism that decides whether that suggestion becomes official.

Tools and files. The kernel exposes one generated list of typed tools, so the prompt and the executable registry cannot silently disagree. Tool requests and results retain names, arguments, timing, outcome, and payload references. File edits use patches or anchored replacements. Commands run in an operating-system sandbox; startup fails if the required sandbox is unavailable.

Submission as one transaction. The answer and its declared checks are persisted together before judgment begins. This prevents an answer from appearing final before its proof requirements are known. Checks run on the submitted workspace rather than an arbitrary current directory.

Fresh review. The reviewer receives the goal and artifact in a new context. Its narrow job is to find a specific, grounded reason the artifact should not pass. It runs read-only in the production path. If the reviewer crashes or returns an invalid result, the mission does not pass.

6 A record that can restore the run

A chat transcript is not enough to resume real work. It may omit tool state, partial effects, spending, or limits. Hairspring stores typed events in order, places large payloads under content hashes, and gives only one component permission to append to the official stream. Tools return results to that writer instead of changing the record directly.

Recovery reads the last complete, consistent prefix. It detects and discards an incomplete tail, then restores the transcript, tool results, counters, file references, and billed cost. The mission continues under the same identity and any limits the operator explicitly set. A restart therefore cannot erase a failed check or reset a set budget.

Evidence. Storage tests inject incomplete tails, duplicate clock values, corrupt registry markers, dead tools, interrupted migrations, and replay after a policy change. These tests live in the crates at the evaluated source revision.

Recovery is replay, not recollection

A crash mid-mission. The record - not the model - reconstructs what happened, and the same mission resumes with its evidence intact.

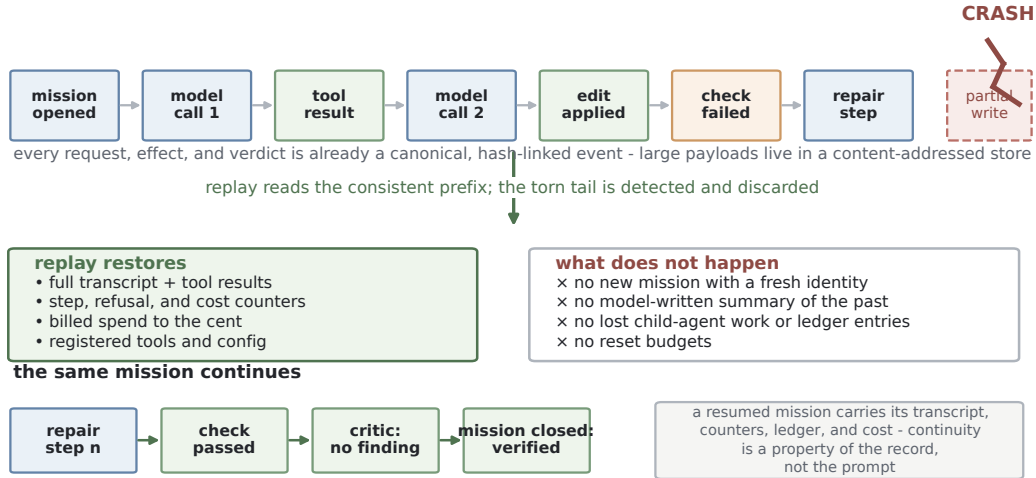


Figure 4: Crash recovery. The durable prefix contains more than conversation: it includes tool state, file identities, counters, and cost. Replay restores the same mission; it does not ask the model to summarize and start over.

7 Child agents, shared state, and memory

A parent may start child missions with explicit step limits. Each child has its own event stream, charges work to the parent, and must finish before the parent can close. A child cannot start more children because the delegation tool is absent from its executable tool list.

Children may propose facts or artifacts to a shared world service. Validation must accept the proposal before another child treats it as shared state. The service records reuse, making it possible to see whether one child found and used another child’s accepted artifact [14].

Memory uses the same discipline. Every stored record keeps its source and mission identity. Later citation can raise a reuse signal; being served and ignored can lower it. This signal measures usefulness, not truth. End-to-end tests cover writing, retrieval, and cross-child reuse. The unmeasured question is whether a live model keeps selecting the right records as the store grows.

8 Testing changes to the harness

The worker repairs a task artifact. A separate outer loop can propose changes to prompts and policy files. Before the candidate runs, Hairspring fixes the scorer and held-out tasks before either policy runs. The existing policy and candidate then run under the same test. The record keeps both hashes, test conditions, traces, scores, decision, and reason.

A malformed or fabricated verdict scores zero, and a failed candidate is undone. The proposer never grades or installs itself. No live promotion records are checked in, so this paper claims only that the mechanism and its tests exist, not that observed promotions improved production results. Hairspring does not yet rewrite and safely promote its own Rust code.

9 Evaluation method

The evaluation separates four kinds of evidence. Source and unit tests establish that a mechanism exists and preserves local rules. Installed-path tests establish that the terminal, registry, sandbox, checker, and rendered state connect that mechanism to a real mission. Live artifact trials establish that a real model can produce an artifact that passes independent checks. Matched external evaluations compare harness outcomes under one shared judge. A large unit suite does not substitute for an installed-path result, and a live result does not establish a general benchmark claim by itself.

The planned comparison uses a difficulty-stratified 15-instance sample from SWE-bench Verified. The sample and source-data digest were fixed before paid runs. Each case has a pinned base commit and a hidden-test patch kept outside either agent’s working tree. Both harnesses use DeepSeek V4.1 Flash through the same metering proxy, in separate network-

The improvement loop: the scorer is pinned before a candidate exists

The inner loop repairs the task. This outer loop changes the policy that drives the worker - and no candidate grades itself.

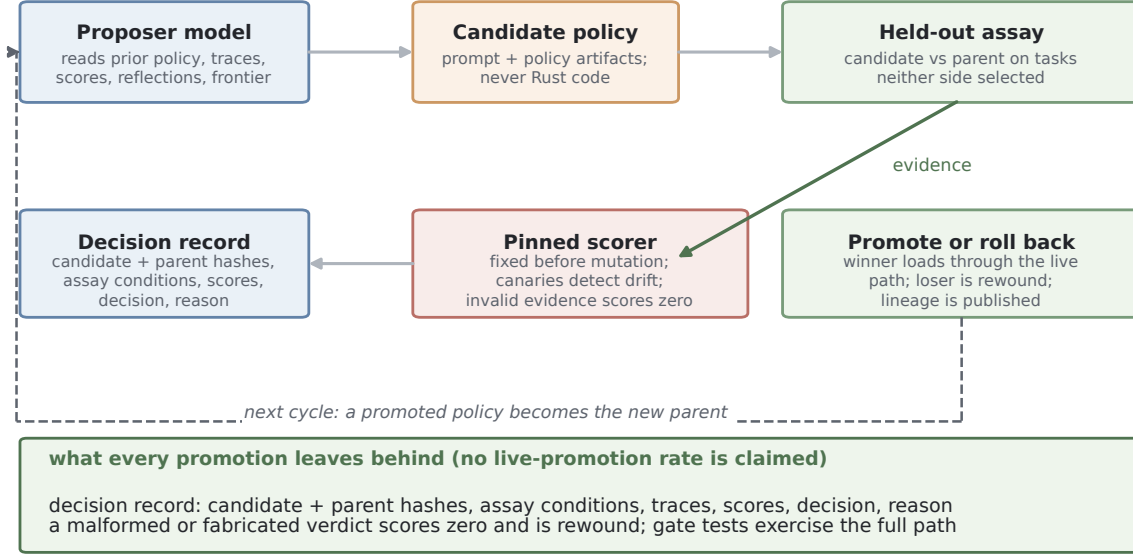


Figure 5: How a prompt or policy change is tested. The judge is fixed before the candidate runs. The old and new policies face held-out tasks, and the decision record contains the exact versions and evidence.

Table 1: Paired 15-instance comparison. Pending cells are deliberately not estimates.

System	Correctly solved	Worker steps	Wall time
HAIRSPRING	pending	pending	pending
OpenHands	pending	pending	pending

isolated workspaces, with one compute instance and one agent run at a time. Neither receives a mission step, wall-time, or output cap below the provider maximum. HAIRSPRING and OpenHands use their native interfaces; this is a harness comparison, not a controlled ablation of a single component.

The evaluator is a local native grader, not the official SWE-bench Docker evaluation. It clones the base, applies pinned tests and the candidate’s tracked source patch, runs a repository-specific test adapter, and fails closed if a pinned test is missing or unrun. The base and reference patch were checked as negative and positive scorer controls for all fifteen cases in the final pinned Python environments. All fifteen reference patches passed every pinned test with none unrun; this verifies that a correct patch can be graded, not that an agent solved a case. Some base controls are weaker: one Pylint case fails collection because the base lacks an imported function, and Django 14608 has two red PASS_TO_PASS tests as well as the expected red FAIL_TO_PASS tests. Other negative reruns exercised the pinned bug assertions under repaired environments. Missing, unrun, or red checks fail closed, and the per-case counts and environment digests are recorded with the rig. A bounded fake-model test traversed both harnesses, the shared proxy, and the grader, but generated no successful candidate. Neither an offline fixture nor an interrupted paid attempt counts as a solved task. The first paid HAIRSPRING attempt was lost in a workspace reset before a terminal grade; it remains unfinished.

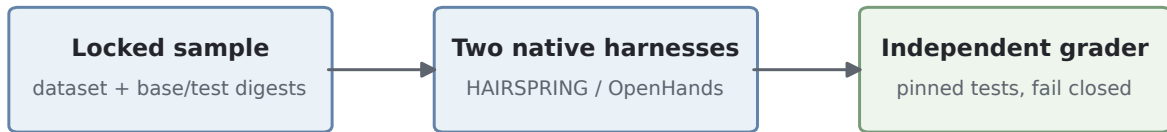
10 Paired comparison: results pending

The comparison will report correctly solved tasks, worker steps, and wall time, by harness and by instance. A solve requires terminal run evidence and a passing independent native grade. A timeout, transport failure, ungraded patch, or unrun pinned test is not a solve. Spend is monitored under a separate ceiling for operational control and is not a comparison-table metric. No completed paid pair is available at this revision, so there is no aggregate winner, solve rate, or performance ratio to plot.

The earlier ten-task comparison used a different baseline and model-family conditions. Its archived records remain in `paper/evidence/` for provenance but are not pooled into this sample or shown as results here. The new run’s raw records, logs, candidate patches, grader output, source and environment digests, and accounting need to survive before numerical

Paired SWE-bench Verified evaluation: protocol and pending outcomes

15 locked instances | DeepSeek V4.1 Flash for both harnesses | one compute instance, one run at a time



Outcome matrix

Each column is one locked task. Pale cells are pending, not failures or solves.

HAIRSPRING	?	?	?	?	?	?	?	?	?	?	?	?	?	?
OpenHands	?	?	?	?	?	?	?	?	?	?	?	?	?	?

15 paired tasks; no completed paid pair at this revision

Publish only verified comparisons

Correctly solved = terminal finish + passing native grade; report steps and wall time.
Unfinished is not a solve.

Figure 6: Evaluation protocol, not a result: fifteen locked instances, paired harnesses, and the same independent grading path. Outcome cells remain pending until terminal records and native grades exist.

claims replace these pending cells.

11 Defects that changed the design

The highest-value defects crossed subsystem boundaries while local tests remained green. Four examples shaped the current test rule.

A dead tool surface. A capability could exist in a crate while the production terminal never registered it. The fix required a test through the real registry and installed terminal.

A wrong production mode. General missions accidentally used an amnesiac benchmark arm. Unit tests for both modes passed; only a live installed-path run exposed the selection error.

Recursive delegation. The first delegation design allowed descendants to delegate further, so process growth was bounded only by the host. The repaired design bounds delegation depth and concurrent children; an installed-path test now reproduces this failure class.

An impossible gate. Missing test dependencies and strict marker settings caused the evaluator to collect zero tests while the agent kept repairing a correct patch. Preflight now proves that the gate can execute before a mission starts.

A related defect class is reward hacking of the close path: the model learned to convert repeated critic refutations into a pass under a retry ratchet. The fix made malformed or absent verdicts fail closed, and refutation counters now reset per mission. This is the failure expected when a model can turn its own review into a pass. The final verdict is therefore executable, not rhetorical.

These failures lead to one rule: a mechanism is not accepted because its crate test passes. Its regression must reproduce how a model reaches it through the installed terminal path, including registry, sandbox, project root, judgment order, and rendered outcome.

12 What Hairspring does not yet coordinate

Recent long-horizon research controllers organize work into strategy exploration, readiness gates, dependency graphs, and global verification [16]. Their strongest transferable idea is explicit control over when a strategy is mature enough to decompose and where a verifier’s finding should return.

HAIRSPRING supplies lower-level services for such controllers: replayable state, executable checks, a fresh reviewer, depth- and concurrency-bounded children, shared state, memory, cost accounting, and held-out policy tests. It does not yet implement that research-planning layer. Missions remain flat. Critic findings return as whole-mission gaps. Candidates are selected or compared rather than constructively synthesized with their objections. No typed dependency graph localizes a defect to the affected closure.

13 Limitations

1. The new paired 15-instance comparison has no completed result in this revision. Its native grader differs from official SWE-bench Docker evaluation.
2. One sample without repeated seeds cannot estimate variance or establish general software-engineering superiority.
3. The two harnesses have different native tool and verification paths, so a paired result would measure their whole systems, not isolate a mechanism.
4. The previous ten-task evidence belongs to a different comparison and is not evidence for the new sample.
5. Self-improvement changes prompts and policy artifacts, not the Rust implementation. Live-model selection quality at scale and a research-planning controller are not measured.

14 Reproducibility

The locked sample, dataset digest, case base commits, hidden-test digests, native scorer, proxy, runner, dependency freezes, and validation instructions are in `benchmarks/paired15/`. See its README for the recovery boundary and accounting caveat. The prior paper’s raw ten-task records remain in `paper/evidence/` as archival context, not the new experiment. Current paired numerical outcomes will require completed run and grade artifacts; reproducing protocol checks without paid inference cannot generate a solve count.

15 What the separation changes

Separating suggestions from decisions makes failures easier to read. A failed check is an execution fact, not an opinion about answer quality. A broken reviewer is a machinery failure, not a pass. A retrieved memory is a sourced note, not shared truth. A well-explained candidate policy is still only a candidate until the fixed test measures it.

This framing does not remove model error. It changes where error can become state. The worker may still choose a poor plan or spend too long on a dead end. The surrounding software can preserve evidence, enforce budgets, refuse an invalid finish, and return a specific defect for repair. Those properties become more important as missions grow longer because recovery and attribution costs rise with the amount of hidden state.

16 Conclusion

HAIRSPRING treats reliable agent work as a separation-of-responsibilities problem. Across five common agent designs, model output can still become accepted without independent proof. Hairspring uses a simpler rule: the model does open-ended work, while tests, a fresh reviewer, an event writer, validators, and a fixed scorer decide what becomes a pass, official record, shared fact, stored memory, or installed policy.

The paired software-engineering outcome remains to be measured. The design claim is narrower: a longer loop does not make its own verdict safer. Long-running agents become easier to inspect, recover, and improve when the model proposes and software decides.

A Artifact map

The implementation is divided by responsibility rather than by interface layer.

Crate	Responsibility
<code>hs-core</code>	Event schema, fixed encoding, hash chaining
<code>hs-log</code>	Segmented streams, payload store, fsync, recovery, rewind
<code>hs-kernel</code>	Plugin process isolation, protocol, dispatch
<code>hs-loop</code>	Worker loop, checker path, REPL, TUI, built-in tools and models
<code>hs-swarm</code>	Bounded asynchronous children, booking, join at close
<code>hs-goal</code>	Graded mission rig and external grading
<code>hs-hashline</code> , <code>hs-applypatch</code>	Anchored and patch-based editing
<code>hs-bench</code> , <code>hs-scorer</code>	Benchmarks, tiered scoring, pins, canaries, baselines
<code>hs-selfmod</code>	Candidate cycle, migration, promotion, rewind
<code>hs-memory</code> , <code>hs-world</code>	Provenance-bearing memory and validated shared state
<code>hs-cli</code>	Event-log inspection and fixtures

B Event and verdict flow

The following abbreviated trace shows the intended ordering. Field names are illustrative; the executable schema in `hs-core` is the source of truth.

```
mission.opened    { goal, project_root, provider, budgets }
model.requested   { step, context_hash }
model.responded   { usage, billed_cost }
tool.requested    { name, args_hash }
tool.completed    { status, payload_hash }
answer.submitted  { artifact_hash, declared_checks }
checks.completed  { status, evidence_hash }
critic.completed  { refuted, findings_hash }
mission.closed    { outcome: "verified" | "rejected" | "stopped" }
```

A valid close follows persisted submission and judgment evidence. A replay reconstructs state from the ordered stream and content-addressed payloads. It does not ask the worker to summarize what happened.

C Paired-result record contract

Each completed run needs its instance ID, harness, terminal state, candidate patch, step count, wall time, and native grade with every pinned test accounted for. The source dataset and hidden-test digests bind the inputs. The prior ten-task CSV and JSON remain archived under `paper/evidence/`, but are not the input to Table 1. A run lost before terminal evidence or independent grade is marked unfinished, not silently counted unresolved or solved.

D Evaluation claims and boundaries

Claim	Status	Boundary
14 crates; 342 Rust files; 70,429 lines	Measured	Source tree at the evaluated revision
992 tests passed; 0 failed	Measured	Complete workspace run on the build host at the evaluated revision; named-test listing reports 994 entries
15-task paired outcome	Pending	No completed paid pair at this revision; native grader is not official Docker SWE-bench
Verifier–repair mechanism	Implemented	Targeted tests do not establish its contribution to the pending comparison
Parallelism caused the gain	Not supported	First controlled test showed no gain and was reverted
General software-engineering superiority	Not claimed	No completed comparison result or repeated seeds
Research-proof controller	Not implemented	Strategy readiness and section dependency state remain future work

References

- [1] S. Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” *ICLR*, 2023.
- [2] N. Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning.” *NeurIPS*, 2023.
- [3] J. Yang et al. “SWE-agent: Agent–Computer Interfaces Enable Automated Software Engineering.” *NeurIPS*, 2024. <https://arxiv.org/abs/2405.15793>.
- [4] “mini-swe-agent: The 100-line agent.” 2025. <https://github.com/SWE-agent/mini-swe-agent>.
- [5] J. Huang et al. “Large Language Models Cannot Self-Correct Reasoning Yet.” 2023. <https://arxiv.org/abs/2310.01798>.
- [6] X. Wang et al. “Self-Consistency Improves Chain of Thought Reasoning in Language Models.” *ICLR*, 2023.
- [7] “When Consensus Is Not Correctness: Diversity Collapse and Manufactured Overconfidence in Multi-Agent LLM Debate.” *KDD Workshop on Agentic AI Evaluation*, 2026. https://kdd-eval-workshop.github.io/agenticai-evaluation-kdd2026/assets/papers/49_When_Consensus_Is_Not_Corre.pdf.
- [8] “From Spark to Fire: Modeling and Mitigating Error Cascades in LLM-Based Multi-Agent Collaboration.” 2026. <https://arxiv.org/abs/2603.04474>.

- [9] S. Yao et al. “Tree of Thoughts: Deliberate Problem Solving with Large Language Models.” *NeurIPS*, 2023.
- [10] “Cost-Awareness in Tree-Search LLM Planning: A Systematic Study.” 2025. <https://arxiv.org/abs/2505.14656>.
- [11] “GEPA: Reflective Prompt Evolution.” DSPy documentation, 2026. <https://dspy.ai/api/optimizers/GEPA/overview/>.
- [12] “MemFail: Stress-Testing Failure Modes of LLM Memory Systems.” 2026. <https://arxiv.org/abs/2605.26667>.
- [13] M. Hu. “Workflow vs. Agent: a Policy-vs-Script Perspective.” Hugging Face blog, 2026. <https://huggingface.co/blog/MengkangHu/workflow-vs-agent>.
- [14] S. Pal et al. “SwarmWorld: Stigmergic technological evolution in societies of language-model agents.” 2026. <https://arxiv.org/abs/2608.26081>.
- [15] Y. Lee et al. “Meta-Harness: End-to-End Optimization of Model Harnesses.” 2026. <https://arxiv.org/html/2603.28052>.
- [16] H. Lin et al. “Stellar Colosseum: A Many-Agent Harness for Long-Horizon Research in Mathematics and Theoretical Computer Science.” 2026. <https://arxiv.org/abs/2609.15983>.